

The OKS Configuration Database & Query Language

A Consolidated Technical Reference for the ATLAS / DUNE-DAQ
Object Kernel Support (OKS) System

Compiled from the DUNE-DAQ **oks** and **dal** source repositories, the original CERN ATLAS TDAQ GitLab sources, published papers, CERN Indico tutorials, shifter documentation, and a curated gold-standard set of English → OKS-query training pairs.

Prepared for: Minor Project — CERN-focused Intelligent Agent for Physics Data Queries (Asmit, B.E. Computer Engineering, Roll No. 080BCT017)

Harvest / compilation date: 2026-08-08

Table of Contents

1. Executive Summary
2. What is OKS? — Conceptual Overview
 - 2.1 OKS in the ATLAS / DUNE-DAQ Architecture
 - 2.2 Core Concepts: Classes, Objects, Attributes, Relationships
 - 2.3 Storage Model — Schema Files and Data Files (XML)
 - 2.4 Editors and Tooling
3. The OKS Query Language
 - 3.1 Grammar (BNF)
 - 3.2 Reserved Tokens and Comparators
 - 3.3 Scope: 'this' vs 'all'
 - 3.4 Attribute Comparisons
 - 3.5 Logical Composition — and / or / not
 - 3.6 Relationship Expressions — some / all
 - 3.7 Object-ID Comparisons
 - 3.8 Path Queries
 - 3.9 The oks_dump Command-Line Tool
4. The C++ / Python OKS Library (DUNE-DAQ/oks)
5. The DAL Package — Typed Access Layer (DUNE-DAQ/dal)
 - 5.1 The Three-Tier Tutorial Example
 - 5.2 ConfigVersion — 'Which Configuration Is Running?'
 - 5.3 Core Schema Class Hierarchy
6. Configuration Versioning and Git-Backed Repositories
7. Worked Schema and Data Examples (XML)
8. Gold-Standard English → OKS-Query Pairs (Corpus Sample)
9. Academic and Technical Literature
 - 9.1 CHEP 2008 Configuration Service Paper
 - 9.2 L1Calo Database User Guide
 - 9.3 Papers Not Retrievable
10. CERN Indico Tutorials and Talks
11. Original CERN GitLab Sources
12. Shifter-Facing Documentation
13. Source Access Status (Captured vs Blocked)

14. Final Deliverables Summary and Corpus Statistics
15. Appendix A — Full Reserved Token / Comparator Reference
16. Appendix B — Provenance Map

1. Executive Summary

This document consolidates an entire web/source-code harvest built around the **OKS (Object Kernel Support)** configuration database used by the ATLAS experiment's Trigger/DAQ system and its modern descendant used by DUNE-DAQ. The original goal of the harvest was to build a training corpus for an English-question → OKS-query fine-tuning dataset (see Section 8), but the underlying material is a complete technical picture of OKS: its C++/Python library, its query grammar, the DAL typed-access layer built on top of it, its XML-based schema/data storage model, its git-backed versioning system, and the operational (shifter-facing) documentation that explains how physicists actually use it during data-taking.

Eleven source files were produced during the harvest and are summarised, in full, in this report: six numbered notes files (01 through 06) covering source code, DAL, papers, Indico talks, GitLab mirrors and shifter manuals; a final report; an access-status ledger; a 50-row gold-standard JSONL corpus of question/query pairs; an XML file of annotated schema/data examples; and a compact query-examples reference. Nothing from the original files has been omitted — every table, code snippet, and fact recorded in the source notes is reproduced or summarised below, organised conceptually rather than file-by-file so the whole system can be understood end-to-end.

Layer	What it is	Key artifact(s)
OKS engine (C++)	Generic object manager: classes, objects, attributes, relationships, query engine, git-backed versioning	include/oks/*.hpp, src/query.cpp
DAL (typed layer)	Auto-generated typed C++/Python classes over an OKS schema; Config/ConfigObject API	dal/core.schema.xml, tutorial.py
Schema / Data files	Human-readable XML; schema defines classes, data files hold concrete objects	*.schema.xml, *.data.xml
Query language	Lisp-like string grammar: attribute/relationship/object-id comparators, and/or/not, path queries	(all ("Name" "x" =))
Versioning	Git-backed config repository; tag / hash / date selectors; used-config published to Information Service	oks_clone_repository --version tag:r380689@all_hosts
Tooling	oks_dump (CLI query/dump), OKS Data Editor (GUI), DBE (DataBase Editor)	apps/oks_dump.cxx

2. What is OKS? — Conceptual Overview

2.1 OKS in the ATLAS / DUNE-DAQ Architecture

OKS stands for **Object Kernel Support**. It is the configuration database used by the ATLAS DAQ/HLT (Trigger and Data Acquisition) system and, in a forked and modernised form, by the DUNE-DAQ collaboration. The configuration service is part of the broader DAQ + Detector Controls stack; its job is to answer, for every run, questions such as: which parts of the detector participate; when and where processes are started; what run-time environment and parameters each process needs; how the system should recover from errors; and in what order the system should shut down. It is accessed concurrently by thousands of processes at run start, supports notification and reconfiguration of mid-run changes, and archives every configuration actually used so that it can be browsed later by experts or replayed during offline event processing (CHEP 2007 paper, J. Almeida et al., see Section 9.1).

2.2 Core Concepts: Classes, Objects, Attributes, Relationships

OKS is described in the CHEP paper as an **object manager**: 'the basic entity is an object with a unique object identifier.' Around that idea sit four building blocks:

- **Classes** — schema-level definitions of a type of object (e.g. Application, Computer, Partition). Classes can be abstract, and support single/multiple inheritance via `<superclass>`.
- **Attributes** — typed fields on a class (string, enum, integer types such as `u16`, `float`, ...), each with an optional default (`init-value`), a value range, and an `is-not-null` constraint. Attributes may be single- or multi-valued.
- **Relationships** — typed references from one object to another (or to a set of others), with cardinalities (`low-cc/high-cc`: `zero/one/many`) and flags for composite or exclusive ownership.
- **Objects** — concrete instances of a class, identified by a unique object-id, holding actual attribute values and relationship references. Classes/objects can be created and modified dynamically at runtime.

Integrity constraints, composite objects (an object that owns and is destroyed together with its children) and change notifications are native features of the engine (CHEP 2007 paper, lines 656–685).

2.3 Storage Model — Schema Files and Data Files (XML)

OKS databases are stored as human-readable **XML files**, split into two kinds: `*.schema.xml` files (class definitions) and `*.data.xml` files (concrete objects). Files can `<include>` other files, which is how a small site-specific schema (e.g. an L1Calo detector schema) can subclass a shared 'core' schema without modifying it. The current data-file format is v2.2 (the 'value inside tag' layout, introduced tdaq-08-03-01) — see the worked examples in Section 7.

For scale and long-term archiving, relational back-ends are also supported via CORAL (Oracle, MySQL, SQLite), oriented toward incremental archiving, plus RDB CORBA servers for remote access with query-result caching on the server side (CHEP 2007 paper, lines 689–753).

2.4 Editors and Tooling

Two graphical tools are described across the sources: a UML-like **Schema Editor** for designing classes, and a **Data Editor** for browsing and editing objects, which includes a graphical query constructor whose output can be saved as a reusable query file and later replayed through the editor or the OKS API (CHEP 2007 paper, lines 766–776). In DUNE-DAQ the same role is played by the **DBE (DataBase Editor)**, a run-control GUI documented on

ReadTheDocs, which additionally exposes check-in/check-out/commit semantics for the underlying git repository (see Section 12).

The original ATLAS oks_utils package ships the full 'OKS Data Editor' (Motif-based) online help tree, captured locally as 54 HTML files (including screenshots) covering the Main Window, Class Window, Data File Window, Object Window, Graphical Window, Replace Window, Message Log Window, and — most relevant to this report — the **Query Window**, whose content is the authoritative user-level description of how a query is built interactively (see Section 3).

3. The OKS Query Language

The query language lets any tool (the C++ API, oks_dump, the Data Editor's graphical query constructor, or Python bindings) ask OKS for 'all objects of a class (optionally with subclasses) whose attributes and/or relationships satisfy a boolean expression.' Its authoritative source is src/query.cpp (reserved tokens, lines 23–39) together with the comment block in include/oks/query.hpp (~lines 300–420) and the CERN RELEASE_NOTES.md, which documents when features such as the regex comparator and path queries were introduced. Query files are plain text and Lisp-like, and are simple enough to be hand-edited: the Query Window help states, 'the format of the query file is simple and can be edited manually by any text editor.'

3.1 Grammar (BNF)

```
query      ::= "(" scope expr ")"
scope      ::= "this" | "all"           ; this = class only, all = class + subclasses
expr       ::= attr_cmp | uid_cmp | rel_expr | and_expr | or_expr | not_expr
and_expr   ::= "and" expr+
or_expr    ::= "or" expr+
not_expr   ::= "not" expr
attr_cmp   ::= "(" "\"" attr-name "\" " "\"" value "\" op ")"
uid_cmp    ::= "(" "object-id" "an-object-id" "=" ")"
rel_expr   ::= "(" "\"" rel-name "\" ("some"|"all") expr ")"
path_query ::= (path-to "\"" dest-id@class "\" (direct|nested "\"" rel "\" ...))
op         ::= "=" | "!=" | "~=" | "<" | "<=" | ">" | ">="
```

This grammar is reconstructed from prose documentation (there is no formal grammar file in the repositories) and is deliberately labelled as such in the underlying corpus — the README, the Query Window HTML help and the RELEASE_NOTES are the ground truth.

3.2 Reserved Tokens and Comparators

Verbatim from src/query.cpp, lines 23–39:

```
const char * OR      = "or";
const char * AND     = "and";
const char * NOT     = "not";
const char * SOME    = "some";
const char * THIS_CLASS = "this";
const char * ALL_SUB  = "all";           // in CERN sources: ALL_SUBCLASSES
const char * OID     = "object-id";
const char * EQ      = "=";
const char * NE      = "!=";
const char * RE      = "~=";
const char * LE      = "<=";
const char * GE      = ">=";
const char * LS      = "<";
const char * GT      = ">";
const char * PATH_TO = "path-to";
const char * DIRECT  = "direct";
const char * NESTED  = "nested";
```

The C++ query classes that implement this grammar (declared in include/oks/query.hpp) are: OksQuery, OksQueryExpression, OksComparator, OksAndExpression, OksOrExpression, OksNotExpression, OksRelationshipExpression, and oks::QueryPath. The comparator enum values map to the text operators as follows:

C++ enum (OksQuery::)	Text operator
equal_cmp	=
not_equal_cmp	!=

less_cmp	<
less_or_equal_cmp	<=
greater_cmp	>
greater_or_equal_cmp	>=
(regex comparator, index.hpp)	~=

Note: the regex comparator '~=' was added in tdaq-01-09-00; it did not exist in the earliest OKS releases.

3.3 Scope: 'this' vs 'all'

Every query wraps its expression in a scope marker: (this <expr>) restricts matching to objects of exactly the named class, while (all <expr>) matches the class and every one of its subclasses. Queries are also described in the Query Window help as 'weak-typed': an OKS query does not strongly depend on class type — if two different classes happen to share an attribute name, the same query text can, in principle, be applied to both.

3.4 Attribute Comparisons

A comparator compares a named attribute against a quoted string value using one of the seven operators above. Worked C++ example (oks_utils/examples/comparator.cpp):

```
#include <oks/attribute.h>
#include <oks/query.h>

int main()
{
    const OksAttribute a("Name", OksAttribute::string_type, false, "", "unknown", "...", true);
    OksComparator qc(&a, new OksData("Peter"), OksQuery::equal_cmp);
    std::cout << qc << std::endl;    // prints  "Name" "Peter" =
```

Full class-level query, from oks_utils/examples/query.cpp — building a small 'Car' schema and finding cars with Max Speed > 200:

```
OksKernel k;
k.new_schema("/tmp/car.schema");
k.new_data("/tmp/car.data");
OksClass * p = new OksClass("Car", "Describes a car", false, &k);
OksAttribute * a = new OksAttribute("Max Speed", OksAttribute::u16_int_type,
                                     false, "", "160", "...", true);

p->add(a);
OksObject * bmw316i = new OksObject(p, "BMW 316i");
OksObject * bmw318i = new OksObject(p, "BMW 318i");
OksObject * bmw320i = new OksObject(p, "BMW 320i");
OksData d((uint16_t) 196);  bmw316i->SetAttributeValue("Max Speed", &d);
d.Set((uint16_t) 201);      bmw318i->SetAttributeValue("Max Speed", &d);
d.Set((uint16_t) 214);      bmw320i->SetAttributeValue("Max Speed", &d);

// "find all cars with Max Speed > 200"
OksQuery q(false, new OksComparator(a, new OksData((unsigned short) 200),
                                   OksQuery::greater_cmp));
std::list<OksObject *> * result = p->execute_query(&q);
```

English-string equivalent, as used in the training corpus: (all ("Max Speed" "200" >)).

3.5 Logical Composition — and / or / not

Boolean composition uses the OksAndExpression, OksOrExpression and OksNotExpression classes, illustrated in examples/and_expression.cpp, or_expression.cpp and not_expression.cpp:

```
OksAttribute a("Height", OksAttribute::float_type, ...);
OksOrExpression or_q;
```



```

or_q.add(new OksComparator(&a, new OksData((float)1.65), OksQuery::greater_or_equal_cmp));
or_q.add(new OksComparator(&a, new OksData((float)1.88), OksQuery::less_or_equal_cmp));

OksNotExpression ne(new OksComparator(&a, new OksData((unsigned short)25),
                                     OksQuery::less_cmp));

```

Textual form: 'anything with height ≥ 1.88 or ≤ 1.65 ' becomes (or (all ("Height" "1.65" $\geq$)) (all ("Height" "1.88" $\leq$))); 'adults (age not < 18)' becomes (not (all ("Age" "18" $<$))).

3.6 Relationship Expressions — some / all

A relationship expression asks whether the objects referenced through a named relationship satisfy a nested query, with two quantifiers: *some* (at least one referenced object matches) and *all* (every referenced object matches). C++ form (examples/r_expression.cpp):

```

OksRelationship r("has car", "Car", OksRelationship::Zero, OksRelationship::Many, ...);
OksRelationshipExpression rqe(&r, new OksComparator(&a, new OksData("BMW"),
                                                    OksQuery::equal_cmp), false);

```

Query-string form, e.g. 'all applications, including subclasses, running on host lxplus001': (all ("RunsOn" some (object-id "lxplus001.cern.ch" =))).

3.7 Object-ID Comparisons

An object can be matched by its unique object-id using the special *object-id* pseudo-attribute, which the grammar restricts to the *=* operator only — (object-id "an-object-id" =). This is normally nested inside a relationship expression, e.g. finding which objects reference object 'Z' through relationship 'R': (all ("R" some (object-id "Z" =))) (documented in CERN RELEASE_NOTES tdaq-01-02-00).

3.8 Path Queries

A separate, more powerful query form — the **path query** — asks OKS to find a chain of relationships connecting a starting object to a destination object or class of objects, introduced in RELEASE_NOTES tdaq-01-02-00. Syntax:

```

(path-to "destination-object" (direct "rel-name" ["rel-name"*] [(nested "rel" ...)]))

```

Worked example from the release notes:

```

(path-to "my-id@my-class" (direct "A" "B" (nested "N" (direct "X" "Y" "Z"))))

```

Meaning: the start object must have relationships A and B; the object(s) referenced via A/B must have relationship N; the object(s) reached via N must have relationships X, Y, Z; the destination counts as found if it is reachable through X, Y or Z. A full CLI invocation (also from RELEASE_NOTES tdaq-01-02-00):

```

oks_dump --path "onlsw_test_3x3_lxplus@Partition" \
  '(path-to "lxplus-3x3-21-ctrl@RunControlApplication"
    (direct "Segments" "OnlineInfrastructure" (nested "Segments"
      (direct "Applications" "IsControlledBy" "Resources"))))' \
  daq/partitions/lxplus_tests.data.xml

```

3.9 The oks_dump Command-Line Tool

oks_dump (source: apps/oks_dump.cxx) is the primary command-line interface to the query engine — it is effectively the 'natural text interface' to OKS queries referenced throughout the corpus. Usage banner (lines ~44–81):

```

oks_dump [options] database-file(s)
  [--files-only | --files-stat-only | --schema-files-only ... | --data-files-only | ...]
  [--class class-name [--query query [--print-references depth class... | --print-referenced_by]]]
  [--path object-from object-to query]

```

```

-f | --files-only          print list of oks file names
-F | --files-stat-only ...
-s | --schema-files-only ...
-S | --schema-files-stat-only
-d | --data-files-only
-D | --data-files-stat-only
-c | --class class-name    dump given class (all objects or matching query)
-q | --query query         print objects matching query (can be used with --class)
-r | --print-references N [C1...] print referenced objects
-b | --print-referenced_by [name] print referencing objects
-p | --path obj query      print path from object 'obj' to object of query expression
-i | --input-from-files    read file list from file
-a | --allow-duplicated-objects-via-inheritance

```

Exit codes:

Code	Meaning
0	OK
1	Bad command line
2	Bad OKS file(s)
3	Bad query
4	Class not found
5	Dangling references found

A query file's values are always quoted strings; type interpretation (numeric vs string) is weak and only matters at evaluation time. Parse failures raise oks: : bad_query_syntax.

4. The C++ / Python OKS Library (DUNE-DAQ/oks)

Source repository (public, default branch develop): <https://github.com/DUNE-DAQ/oks> (clone: `git clone https://github.com/DUNE-DAQ/oks`). The DUNE-DAQ fork is baselined on CERN tag oks-08-03-04 (April 2022) — the same engine used by ATLAS TDAQ release tdaq-09-04-00.

4.1 Repository Layout

Path	Content
include/oks/query.hpp	Query classes: OksQuery, OksQueryExpression, OksComparator, OksAndExpression, OksOrExpression, OksNotExpression, OksRelationshipExpression, oks::QueryPath
include/oks/index.hpp	OksIndex (fast object lookup) + query operators
include/oks/kernel.hpp	OksKernel — load/save schema & data, git repository versions
include/oks/class.hpp	OksClass — execute_query(), find_object(), get_object()
include/oks/object.hpp	OksObject — attributes, relationships, find_path()
include/oks/attribute.hpp	OksAttribute — types, ranges, defaults
include/oks/relationship.hpp	OksRelationship — cardinalities, composite/exclusive
include/oks/file.hpp	OksFile — data/schema files, includes, comments
src/query.cpp	Query string parser (the OKS query grammar)
apps/oks_dump.cxx	CLI: dump classes, objects, execute queries, path queries
apps/oks_clone_repository.cxx	Clone git config repository (-b/-c/-t/-d options)
apps/oks_check_schema.cxx, oks_validate_repository.cxx	Schema/repository validation
scripts/oks-*.sh	oks-commit.sh, oks-tag.sh, oks-checkout.sh, oks-import.sh, oks-copy.sh, ...
pybindsrc/module.cpp	Python bindings (module 'oks'); python/oks/__init__.py is intentionally empty — C++ bindings only, built with cmake+pybind11

4.2 Git-Repository Versioning API

Declared in include/oks/kernel.hpp, this is the programmatic side of 'which configuration ran a given run':

- struct OksRepositoryVersion (~line 514): members include tag, sha1, date/time, branch, author, comment, workdir.
- OksKernel::load_file(...), load_schema(...), load_data(...).
- get_repository_root() / get_repository_mapping_dir() (~lines 938, 960).
- tag_repository(tag) (line 1579) — tag the current version, e.g. r380689@all_hosts.

- `get_repository_versions(skip_irrelevant, command_line)` (line 1667) — list all versions; `get_repository_versions_by_hash(sha1)` (1681); `get_repository_versions_by_date(...)` (1695); `read_repository_version(...)` (1705); `get_repository_versions_diff(sha1a, sha1b)` (1643).

Relevant environment variables (docs/README.md; CERN RELEASE_NOTES tdaq-09-01-00): TDAQ_DB_REPOSITORY (enable git), TDAQ_DB_USER_REPOSITORY, TDAQ_DB_VERSION (hash:<sha1> or date:<ts>), OKS_REPOSITORY_MAPPING_DIR, TDAQ_DB_PATH.

4.3 Shifter-Relevant Command Mapping

Question	OKS answer
What config version ran for run 380689?	<code>tag r380689@all_hosts;</code> <code>oks_clone_repository --version tag:r380689@all_hosts</code>
Which version by hash?	<code>oks_clone_repository --version hash:6800fe3b</code>
Which version by date?	<code>oks_clone_repository --version</code> <code>'date:2020-08-02 10:00:00'</code>
List all versions?	<code>oks_dump -r ... / kernel get_repository_versions(...)</code>

Licensing: all OKS/DAL code files are © CERN (ATLAS Collaboration), distributed under a BSD-3-Clause style licence — attribution should be kept when redistributing snippets.

5. The DAL Package — Typed Access Layer (DUNE-DAQ/dal)

Source repository: <https://github.com/DUNE-DAQ/dal> (default branch develop). DAL — 'Data Access Library' — is the typed C++/Python layer built on top of OKS: classes defined in an OKS schema are mapped to generated C++ classes, and objects are read and modified through ConfigObject/Config interfaces rather than the raw OKS API. The original ATLAS implementation and its documentation live in the same CERN GitLab family described in Section 11.

5.1 Repository Layout (Relevant Files)

Path	Content
docs/README.md	"An Introduction to OKS" — the canonical user-level doc
docs/TWiki.txt	924-line copy of the old ATLAS TWiki OKS pages (DaqHltOks)
docs/RELEASE_NOTES.md	DAL release notes (migrated from ATLAS tdaq)
include/dal/ConfigVersion.hpp	ISInfo w/ Version field — publish used config version to IS
apps/dal_get_config_version.cxx	CLI: publish config version into IS (-p partition)
apps/dal_set_config_version.cxx	CLI: set config version in IS (-v, -p, -r)
apps/dal_dump_apps.cxx (+ friends)	Dump running applications from config
schema/dal/core.schema.xml	83,540 B / 660 lines — classic TDAQ core schema (Partition, Application, Computer, Segments, ...)
schema/dal/tutorial.schema.xml	Minimal 3-class schema used by the tutorial
scripts/dal_testing.data.xml	Data file matching the tutorial schema
scripts/dal_testing.py / tutorial.py	Python tutorial
CMakeLists.txt	Build config (C++17, pybind11, cmake)

5.2 The Three-Tier Tutorial Example

Tutorial schema (repos/tutorial.schema.xml, 3 classes):

```
<class name="Application" description="A software executable" is-abstract="yes">
  <attribute name="Name" type="string" init-value="Unknown" is-not-null="yes"/>
</class>

<class name="ReadoutApplication" description="An executable which reads out subdetectors">
  <superclass name="Application"/>
  <attribute name="SubDetector" type="enum" range="PMT,WireChamber" init-value="WireChamber"/>
</class>

<class name="RCApplication" description="An executable which allows users to control datataking">
  <superclass name="Application"/>
  <attribute name="Timeout" type="u16" range="1..3600" init-value="20" is-not-null="yes"/>
  <relationship name="ApplicationsControlled" description="Applications RC is in charge of"
    class-type="Application" low-cc="one" high-cc="many"/>
</class>
```

Matching data (repos/dal_testing.data.xml, excerpt):

```
<oks-data>
```

```

<include><file path="tutorial.schema.xml"/></include>
<obj class="RCApplication" id="rc_negative">
  <attr name="Name" type="string" val="NegativeRC"/>
</obj>
...
<obj class="ReadoutApplication" id="0x1" >
  <attr name="Name" type="string" val="Collab"/>
  <attr name="SubDetector" type="enum" val="PMT"/>
</obj>
</oks-data>

```

Python usage (scripts/tutorial.py, verbatim, 1,274 B):

```

import dal.tutorial as tut  # DAL classes generated from tutorial.schema.xml
import dal.core as core

# open the configuration database
f = tut.Config("file:filename=dal_testing.data.xml")
obj = f.get('ReadoutApplication')
print(f"ReadoutApplication: {obj}")    # prints id, attributes, relationships
...
f.set('Timeout', 20)                  # modify an attribute
f.commit()                            # save all changes back to XML files

```

5.3 ConfigVersion — 'Which Configuration Is Running?'

From include/dal/ConfigVersion.hpp: in the OKS schema, class ConfigVersion has a string attribute Version, published to the Information Service under /<partition>/ConfigInfos/.... The Version string encodes the OKS git SHA-1 of the configuration in use, e.g. hash:6800fe3b63a18859ef688612b44e051e4f36e345.

Two CLI apps operationalise this: dal_get_config_version.cxx prints the currently published config version for a partition (-p <partition>) — i.e. 'which configuration am I running?'; dal_set_config_version.cxx sets it (-v <version+sha1> -p <partition> -r).

5.4 Core Schema Class Hierarchy

The full ATLAS (TDAQ) class hierarchy in core.schema.xml runs to hundreds of classes, rooted at: Partition (top), Segment/OnlineSegment, Application / InfrastructureApplication / CustomLifetimeApplication, ComputerBase > Computer/ComputerSet (Platform, HW_Object), Resource, Parameter, SW_Object, HW_Object, Detector, and many more. A representative usage snippet (also found in the CHEP 2008 paper, Section 9.1):

```

#include <dal/Partition.h>
dal::Partition * p = db.getPartition("test");
std::string default_host = p->getDefaultHost();

```

Documentation pointers: docs/README.md gives the 'Introduction to OKS' (schema/classes, objects, attributes, relationships, multi-value/composite/exclusive relationships, data+schema files, init values, ranges, archiving). The DUNE ReadTheDocs mirrors are public: dune-daq-sw.readthedocs.io/en/latest/packages/dal/RELEASE_NOTES.html and the v4.1.0 branch, which was confirmed accessible and mirrors the ATLAS release notes (tdaq-09-01-00, tdaq-08-03-00, ...). The DBE user pages live on a readthedocs branch at .../johnfreeman-update_documentation_instructions/packages/dbe/.

6. Configuration Versioning and Git-Backed Repositories

One of the most operationally important OKS features is that the configuration database itself is a **git repository**. Every 'used configuration' is identifiable by a git commit hash, an optional tag, or a date, letting shifters and offline analysers answer 'exactly which configuration produced this run's data?' with a reproducible artifact rather than a loose description.

Facts pinned down from the CERN RELEASE_NOTES.md (pulled from GitLab tags, see Section 11):

- **tdaq-13-01-00** (current master docs): repository versions addressed by tag, commit hash, or date (e.g. '2026-06-15', '2 years 1 day 3 minutes ago'), or branch; `OksKernel::import_file(to, from)`.
- **tdaq-12-00-00**: `update_data()` keeps existing file layout; `save_data()` rewrites it.
- **tdaq-11-02-00**: ordered multi-value attributes/relationships (sorted on save); aligned commit flushing.
- **tdaq-09-02-01**: postponed changes via git branches; `oks_clone_repository -b <branch> --version tag:...|hash:...|date:...;` the run-number database itself stores the used config version — `rn_ls -c "oracle://atonr_adg/rn_r" ... -a '%xml' returns columns Version = hash:6800fe3... | Config Name = daq/partitions/all_hosts.data.xml;` the git repo is tagged by run number and partition, e.g. `oks_clone_repository --version tag:r380689@all_hosts`.
- **tdaq-09-01-00**: the CVS→git switch; environment variables `TDAQ_DB_REPOSITORY` (git URLs), `TDAQ_DB_USER_REPOSITORY`, `TDAQ_DB_VERSION=hash:...|date:...;`, `OKS_REPOSITORY_MAPPING_DIR`; files are accessed by repository filename, e.g. `oks_data_editor daq/segments/setup.data.xml`; `oks-git-on/off/status` helper functions; a read-only GitLab mirror project at `las-tdaq-oks`.
- **tdaq-08-03-01**: data-file format v2.2 — the 'value inside tag' layout used throughout the worked examples in Section 7.
- **tdaq-01-09-00 ... tdaq-01-02-00**: introduction of the regex ('~=') comparator; path-query syntax; object-id in queries; archiving of `OksSchema + Release` columns; the `oks_ls_data` tool family.

Backend history also records a migration from OWL to Boost date-time handling, and archiving versioning moving from base-only to base+incremental snapshots.

7. Worked Schema and Data Examples (XML)

These five annotated examples come from `oks_schema_examples.xml`, a curated set of copy-ready OKS schema/data snippets with provenance captions, spanning the DUNE-DAQ tutorial, the real ATLAS/TDAQ core schema, the OKS data-type reference, classic DAQ configuration classes, and the offline archiving schema. Together they show the full range of OKS attribute types, relationship styles, and both major on-disk data formats.

7.1 1-tutorial-dal

Minimal 3-class DAL tutorial (DUNE-DAQ/dal): inheritance, enum + integer ranges, default (init) values, is-not-null, and a one-to-many relationship; the matching data file shows real objects. [Sources: <https://github.com/DUNE-DAQ/dal-schema/dal/tutorial.schema.xml>, [scripts/dal_testing.data.xml](https://github.com/DUNE-DAQ/dal-scripts/dal_testing.data.xml)]

Schema file: *tutorial.schema.xml*

Source: <https://github.com/DUNE-DAQ/dal>

```
<?xml version="1.0" encoding="ASCII"?>
<!-- oks-schema version 2.2 -->
<oks-schema>
<info name="" type="" num-of-items="3" oks-format="schema" oks-version="862f2957270"
      created-by="jcfree" created-on="mu2eddaq13.fnal.gov" creation-time="20230123T223700"
      last-modified-by="jcfree" last-modified-on="mu2eddaq13.fnal.gov" last-modification-time="20230123T223700"/>
<class name="Application" description="A software executable" is-abstract="yes">
  <attribute name="Name" description="Name of the executable, including full path" type="string"
    init-value="Unknown" is-not-null="yes"/>
</class>
<class name="ReadoutApplication" description="An executable which reads out subdetectors">
  <superclass name="Application"/>
  <attribute name="SubDetector" description="An enum to describe what type of subdetector it can read out"
    type="enum" range="PMT,WireChamber" init-value="WireChamber"/>
</class>
<class name="RCApplication" description="An executable which allows users to control datataking">
  <superclass name="Application"/>
  <attribute name="Timeout" description="Seconds to wait before giving up on a transition" type="u16"
    range="1..3600" init-value="20" is-not-null="yes"/>
  <relationship name="ApplicationsControlled" description="Applications RC is in charge of"
    class-type="Application" low-cc="one" high-cc="many"/>
</class>
</oks-schema>
```

Data file: *dal_testing.data.xml*

Source: <https://github.com/DUNE-DAQ/dal>

```
<?xml version="1.0" encoding="ASCII"?>
<!-- oks-data version 2.2 -->
<oks-data>
<include><file path="../share/schema/dal/core.schema.xml"/></include>
<obj class="Partition" id="ToyPartition">
  <rel name="OnlineInfrastructure" class="OnlineSegment" id="ToyOnlineSegment"/>
  <rel name="DefaultHost" class="Computer" id="toyhost.fnal.gov"/>
  <rel name="Disabled">
    <ref class="ResourceBase" id="AProblematicResource"/>
  </rel>
</obj>
<obj class="ResourceBase" id="AProblematicResource"/>
<obj class="RunControlApplication" id="ToyRunControlApplication">
  <attr name="InterfaceName" type="string" val="rc/commander"/>
  <attr name="ActionTimeout" type="s32" val="10"/>
  <attr name="IfExitsUnexpectedly" type="enum" val="Error"/>
  <attr name="IfFailsToStart" type="enum" val="Error"/>
  <rel name="Program" class="Binary" id="rc_controller"/>
  <rel name="ExplicitTag" class="Tag" id="ToyTag"/>
</obj>
```



```

<obj class="OnlineSegment" id="ToyOnlineSegment">
  <attr name="T0_ProjectTag" type="string" val="data_test"/>
  <rel name="IsControlledBy" class="RunControlApplication" id="ToyRunControlApplication"/>
  <rel name="InitialPartition" class="Partition" id="ToyPartition"/>
  <rel name="Applications">
    <ref class="CustomLifetimeApplication" id="SomeApp"/>
  </rel>
  <rel name="Segments">
    <ref class="Segment" id="ToyChildSegment"/>
  </rel>
</obj>
<obj class="Computer" id="toyhost.fnal.gov">
  <attr name="HW_Tag" type="enum" val="x86_64-centos7"/>
  <attr name="SW_Tag" type="enum" val="gcc49-opt"/>
  <attr name="NumberOfCores" type="s16" val="2"/>
  <attr name="RLogin" type="string" val="ssh"/>
</obj>
</oks-data>

```

7.2 2-atlas-core-schema

The ATLAS/TDAQ core schema (real production classes). The run-level entry point is the Partition: it holds top-level Segments, the online infrastructure, disabled components, tags, parameters and (important) the database attributes DBPath/DBName/DBVersion — DBVersion is the OKS git SHA used for the partition, i.e. the answer to "what configuration was used". Methods `get_config_version()/set_config_version()` are the DAL interface for that. Also shows the C++/Java dual-language method bodies typical of dal-generated code. [Source: <https://github.com/DUNE-DAQ/dal - schema/dal/core.schema.xml>]

Schema file: *core.schema.xml (excerpt: class Partition)*

Source: <https://github.com/DUNE-DAQ/dal>

```

<class name="Partition" description="The class describes TDAQ configuration including top-level segments, disabled compo
  <attribute name="RepositoryRoot" description="Temporary sw installation area having highest priority" type="string"/>
  <attribute name="IPCRef" description="IPC init reference; default from TDAQ_IPC_INIT_REF env" type="string" init-value=
  <attribute name="DBPath" description="Locations of database repositories (OKS_DB_ROOT); default from TDAQ_DB_PATH" typ
  <attribute name="DBName" description="Name of the OKS data file; default from TDAQ_DB_DATA" type="string" init-value="
  <attribute name="DBVersion" description="Database version (SHA key) if GIT repository is used; default from TDAQ_DB_VE
  <attribute name="DBTechnology" description="Config database technology implementation" type="enum" range="rdbconfig,ok
  <attribute name="RunTypes" type="string" is-multi-value="yes"/>
  <relationship name="Segments" description="Top-level segments composing given partition. To disable a segment, add it
  <relationship name="OnlineInfrastructure" description="Defines online infrastructure segment." class-type="OnlineSegme
  <relationship name="Disabled" description="Lists segments and resources which are temporary disabled." class-type="Com
  <relationship name="DefaultHost" description="Host to run all applications with empty 'RunsOn'." class-type="Computer"
  <relationship name="DefaultTags" description="Default tags used to select binaries for applications without explicit t
  <relationship name="ProcessEnvironment" description="Process environment for any application ran in given partition."
  <method name="set_config_version" description="Set configuration version, i.e. OKS GIT repository SHA. The partition i
    <method-implementation language="c++" prototype="void set_config_version(const std::string& name, bool reload) c
    <method-implementation language="java" prototype="void set_config_version(final String version, boolean reload) thro
  </method>
  <method name="get_config_version" description="Return configuration version used by given partition, i.e. OKS GIT repo
    <method-implementation language="c++" prototype="std::string get_config_version() const"/>
    <method-implementation language="java" prototype="String get_config_version() throws config.GenericException, config
  </method>
</class>

```

7.3 3-all-types

The OKS data type reference (CERN oks repo test): all attribute types (bool..u64, float, double, date, time, string, enum, class), multi-value attributes, hex format, and the compact "data" format (attr val inside tag). Good grounding for translating attribute names to type-correct query values. [Sources: https://gitlab.cern.ch/atlas-tdaq-software/oks - test/all_types.schema.xml, [test/test.data.xml](https://gitlab.cern.ch/atlas-tdaq-software/oks - test/test.data.xml)]

Schema file: *all_types.schema.xml* (excerpt: class Element)

Source: https://gitlab.cern.ch/atlas-tdaq-software/oks/-/raw/master/test/all_types.schema.xml

```
<oks-schema>
<class name="Element" description="It is a class to describe a person">
  <attribute name="Boolean" type="bool"/>
  <attribute name="IntS8" type="s8"/>
  <attribute name="IntU8" type="u8"/>
  <attribute name="IntS16" type="s16"/>
  <attribute name="IntU16" type="u16"/>
  <attribute name="IntS32" type="s32"/>
  <attribute name="IntU32" type="u32"/>
  <attribute name="IntS64" type="s64"/>
  <attribute name="Int64" type="u64"/>
  <attribute name="Float" type="float"/>
  <attribute name="Double" type="double"/>
  <attribute name="String" type="string"/>
  <attribute name="Date" type="date"/>
  <attribute name="Time" type="time"/>
  <attribute name="Enum" type="enum" range="one,two,three,four,five" init-value="one"/>
  <attribute name="Class" type="class" init-value="Container"/>
  <attribute name="ArrayIntS8" type="s8" is-multi-value="yes"/>
  <attribute name="ArrayString" type="string" is-multi-value="yes"/>
  <attribute name="ArrayEnum" type="enum" range="one,two,three,four,five" is-multi-value="yes" init-value="one"/>
  <attribute name="HexIntU8" type="u8" format="hex"/>
  <attribute name="HexIntU64" type="u64" format="hex"/>
</class>
</oks-schema>
```

Data file: *test.data.xml* (excerpt)

Source: <https://gitlab.cern.ch/atlas-tdaq-software/oks/-/raw/master/test/test.data.xml>

```
<oks-data>
<include><file path="all_types.schema.xml"/></include>
<obj class="Element" id="o1-good">
  <attr name="Boolean" type="bool" val="1"/>
  <attr name="IntS8" type="s8" val="-128"/>
  <attr name="IntU16" type="u16" val="50000"/>
  <attr name="IntS64" type="s64" val="-1000000000000"/>
  <attr name="Float" type="float" val="3.1415"/>
  <attr name="Double" type="double" val="2.1928"/>
  <attr name="String" type="string" val="The quick brown fox jumps over the lazy dog."/>
  <attr name="Date" type="date" val="20240809"/>
  <attr name="Time" type="time" val="20240809T150647"/>
  <attr name="Enum" type="enum" val="one"/>
  <attr name="Class" type="class" val="Container"/>
  <attr name="ArrayIntS8" type="s8">
    <data val="1"/>
    <data val="-1"/>
    <data val="127"/>
  </attr>
  <attr name="HexIntU8" type="u8" val="0xff"/>
  <attr name="HexIntU64" type="u64" val="0x123456789abcdef"/>
</obj>
<obj class="Container" id="o1-good-container">
  <attr name="Date" type="date" val="20240829"/>
  <attr name="Time" type="time" val="20240829T104831"/>
  <attr name="Enum" type="enum" val="one"/>
  <attr name="Class" type="class" val="Container"/>
  <rel name="Root" class="Element" id="o1-good"/>
  <rel name="Contains">
    <ref class="Element" id="o1-good"/>
  </rel>
</obj>
</oks-data>
```

7.4 4-daq-conf-schema

Classic DAQ configuration classes (oks_utils tests): software objects, computers, programs, environment variables and the (dynamic) Process class. Shows the old naming conventions still used in production DBs. [Source: https://gitlab.cern.ch/atlas-tdaq-software/oks_utils - tests/DAQ-Configuration.schema.xml]

Schema file: *DAQ-Configuration.schema.xml* (excerpt)

Source:

https://gitlab.cern.ch/atlas-tdaq-software/oks_utils/-/raw/master/tests/DAQ-Configuration.schema.xml

```
<oks-schema>
<class name="Computer" description="Describes an abstract computer.">
  <attribute name="Name" description="Computer name (e.g. pc-tdaq-onl-01)" type="string" is-not-null="yes"/>
  <attribute name="OSType" description="Operating system type" type="enum" range="Linux,SunOS,Win32"/>
  <attribute name="CPUSpeed" description="CPU speed in MHz" type="u16"/>
</class>
<class name="Environment" description="The Environment class is used to describe environment variables and their values.">
  <attribute name="Name" type="string" is-not-null="yes"/>
  <attribute name="Value" type="string"/>
</class>
<class name="Program" description="This class is used to describe platform dependent part of DAQ software component. It is used to describe the command line and the priority.">
  <attribute name="CommandLine" type="string"/>
  <attribute name="Priority" type="s32" init-value="0"/>
</class>
<class name="Process" description="The Process class is used to describe a dynamic physical part of a software object: logical process.">
  <attribute name="Id" type="string" is-not-null="yes"/>
  <attribute name="StartTime" type="time"/>
  <relationship name="Program" class-type="Program" low-cc="one" high-cc="one"/>
  <relationship name="RunsOn" class-type="Computer" low-cc="one" high-cc="one"/>
</class>
</oks-schema>
```

7.5 5-oks2coral-archive

The oks2coral archiving schema: ConfigVersions records which archive (schema/data version) of the OKS database was used for a given run — the offline counterpart of the online "r380689@all_hosts" git tags. [Source: <https://gitlab.cern.ch/atlas-tdaq-software/oks2coral - data/oks-archive-info.xml>]

Schema file: *oks-archive-info.xml*

Source: <https://gitlab.cern.ch/atlas-tdaq-software/oks2coral/-/raw/master/data/oks-archive-info.xml>

```
<oks-schema>
<class name="ConfigVersions" description="The class is used to store archive versions of the OKS database used for given run.">
  <superclass name="Info"/>
  <attribute name="Run" type="u32" is-not-null="yes" init-value="0"/>
  <attribute name="SchemaVersion" type="u32" init-value="0"/>
  <attribute name="DataVersion" type="u32" init-value="0"/>
  <attribute name="TotalVersion" type="string"/>
  <attribute name="Time" type="time" init-value="11-10"/>
</class>
</oks-schema>
```

8. Gold-Standard English → OKS-Query Pairs (Corpus Sample)

The full gold-standard fine-tuning corpus (gold_pairs.jsonl) contains 50 rows in the schema {question, query_oks, note, source_file}, covering every comparator (= != ~= < <= > >=), both scopes (this/all), boolean composition (and/or/not), relationship expressions (some/all), object-id comparisons, nested relationships, path-to queries, git version selectors (tag:/hash:/date:), oks_dump CLI usage and exit codes, and GUI/weak-typing facts. It is grounded in repos/tutorial.schema.xml, repos/test.data.xml, QueryWindow.html and the CERN RELEASE_NOTES; rows marked 'illustrative' are syntax-correct but use invented attribute names for pedagogical variety. All 50 rows are reproduced below in full.

#	Question	OKS Query	Note
1	Which applications are named rc_trigger_1?	(all ("Name" "rc_trigger_1" =))	Attribute comparison with '='. Quoted value string. Scope 'all' = class Application and subclasses.
2	List all applications except rc_trigger_1	(all ("Name" "rc_trigger_1" !=))	'!=' comparison on string attribute Name.
3	Find hosts whose name starts with pc-tdaq-onl	(all ("Name" "pc-tdaq-onl-.*" ~=))	'~=' is regex comparison (added tdaq-01-09-00). Value is an (unquoted-in-pattern) regex string.
4	Get all hosts matching the glob-like pattern pc-tdaq*	(all ("Name" "pc-tdaq.*" ~=))	Translate shell glob to regex: '*' -> '.*'. OKS has no glob operator; only regex '~='.
5	Which readout applications handle PMT detectors?	(all ("SubDetector" "PMT" =))	Enum attribute SubDetector (range PMT,WireChamber). Enum values are still quoted strings in queries.
6	Which readouts are NOT for PMT?	(all ("SubDetector" "PMT" !=))	'!=' on enum attribute.
7	Run-control applications with Timeout greater than 25 seconds	(all ("Timeout" "25" >))	Numeric '>' on u16 attribute Timeout (range 1..3600). Value quoted despite being numeric.
8	Run-control applications with Timeout below 30 seconds	(all ("Timeout" "30" <))	'<' comparison.
9	Applications with Timeout at least 20 seconds	(all ("Timeout" "20" >=))	'>='. 20 is the init-value; boundary included.
10	Applications with Timeout no more than 120 seconds	(all ("Timeout" "120" <=))	'<=' comparison.
11	Apps whose name is not the default 'Unknown'	(all ("Name" "Unknown" !=))	init-value of Name is 'Unknown'; useful sanity filter.
12	Apps named 'Fake' or with Timeout over 500	(or (all ("Name" "Fake" =)) (all ("Timeout" "500" >)))	Logical OR joins two full scoped expressions. Grammar: (or expr expr ...).
13	Applications whose Timeout is NOT less than 25	(not (all ("Timeout" "25" <)))	NOT wraps a whole scoped expression; equivalent to (all ("Timeout" "25" >=)).
14	RC applications that control application rc-readout-1	(all ("ApplicationsControlled" some (all ("Name" "rc-readout-1" =))))	Relationship 'some': at least one referenced object matches the inner query. Inner scope 'all' again includes subclasses.
15	RC applications that control only PMT readouts (every controlled app is PMT)	(all ("ApplicationsControlled" all ("SubDetector" "PMT" =)))	Relationship 'all': EVERY referenced object must satisfy the subquery. Distinguish from 'some' (at least one).
16	Which containers contain object o1-good?	(all ("Contains" some (object-id "o1-good" =)))	object-id comparison, only '=' allowed. Matches Container objects whose Contains list includes the Element with id o1-good.
17	Containers whose Root is exactly o1-good	(all ("Root" some (object-id "o1-good" =)))	Single-valued relationship Root; 'some' works for single refs too.
18	Containers that do NOT contain o1-good	(not (all ("Contains" some (object-id "o1-good" =))))	NOT combined with relationship+object-id.
19	Elements referenced from o1-good-container via Contains	(all ("Contains" some (object-id "o1-good-container" =)))	Note direction: query finds objects whose relationship points AT the given id.

#	Question	OKS Query	Note
20	All applications running on host lxplus001	<code>(all ("RunsOn" some (object-id "lxplus001.cern.ch" =)))</code>	Canonical example from CERN release notes; object-id of a Computer is its hostname.
21	Partitions with an app running on host abc	<code>(all ("Partitions" some (all ("RunsOn" some (object-id "abc" =)))))</code>	Nested relationships: Partitions -> some partition -> RunsOn -> object-id. Each relationship level has its own inner scope.
22	Objects whose 'my-relationship' points to object test	<code>(all ("my-relationship" some (object-id "test" =)))</code>	Generic form from release notes.
23	Objects of class Container with Enum 'one' AND Date 20240827	<code>(and (all ("Enum" "one" =)) (all ("Date" "20240827" =)))</code>	Date stored as ISO string 'YYYYMMDD'; compare with '='.
24	Elements created after 20240827	<code>(all ("Date" "20240827" >))</code>	Dates are ISO strings; lexical > works for YYYYMMDD.
25	Elements with Float value above 3.0	<code>(all ("Float" "3.0" >))</code>	float attribute; value quoted, parsed as double.
26	Elements with Boolean true	<code>(all ("Boolean" "true" =))</code>	bool attribute; data file stores 1/0, but weak typing accepts 'true'/'false' in the query string.
27	Elements whose Class attribute is Container	<code>(all ("Class" "Container" =))</code>	'class' attribute type holds the name of another class; compare with a quoted class name.
28	Elements with an empty ArrayString	<code>(all ("ArrayString" "" =))</code>	Multi-value attribute with empty value list; value is empty string.
29	Containers whose Contains includes at least one Element with IntS32 = -1	<code>(all ("Contains" some (all ("IntS32" "-1" =))))</code>	Relationship + inner attribute query; inner 'all' needed because Contains refs are of class Element (no subclasses here, harmless).
30	Containers holding only elements named like 'o1-'	<code>(all ("Contains" all (all ("Name" "o1-.*" ~=))))</code>	'all' with relationship: every contained element must match. Inner query has no Name attr on Element in test data - illustrative.
31	Only class itself, not subclasses: this scope	<code>(this ("Name" "rc_trigger_1" =))</code>	'this' = exactly the queried class (no subclass expansion); 'all' = class + subclasses.
32	Tutorial cars with Max Speed over 200 (older example)	<code>(all ("Max Speed" "200" >))</code>	Illustrative: original OKS tutorial schema had class Car; attribute names can contain spaces and must be quoted.
33	Containers containing o1-good AND o1-good2	<code>(and (all ("Contains" some (object-id "o1-good" =))) (all ("Contains" some (object-id "o1-good2" =))))</code>	AND of two relationship conditions - both must hold.
34	Containers containing o1-good or o1-good2	<code>(or (all ("Contains" some (object-id "o1-good" =))) (all ("Contains" some (object-id "o1-good2" =))))</code>	OR of two relationship conditions.
35	Apps with Name rc_trigger_1 that are NOT readouts	<code>(and (all ("Name" "rc_trigger_1" =)) (not (all ("SubDetector" "PMT" =))))</code>	Mixed AND + NOT; every branch is a full scoped expression.
36	Hosts running at least one app and none named 'X'	<code>(and (all ("RunsOn" some (all ("Name" ".*" ~=)))) (not (all ("RunsOn" some (all ("Name" "X" =)))))</code>	'.*' regex matches every non-empty string.
37	Query file: the format is plain text	<code>(all ("Name" "foo" =))</code>	Queries can be stored in .query files and loaded from the Query window; format is simple Lisp-like text, editable by hand.
38	Run oks_dump to list all objects of class RCAApplication	<code>oks_dump -c RCAApplication -f tutorial.data.xml</code>	-c selects class, -f selects data file(s). Exit code 0 = OK.
39	Run a query from the command line	<code>oks_dump -c RCAApplication -f tutorial.data.xml -q "(all (\\"Timeout\\" \\"30\\" <))"</code>	-q passes a query string; note escaping of quotes in shell.
40	What do the oks_dump exit codes mean?	0=OK 1=bad CLI 2=bad OKS file 3=bad query 4=class not found 5=dangling refs	Documented mapping; useful for test harness assertions.

#	Question	OKS Query	Note
41	Path query: find RunControlApplication for a partition via OnlineInfrastructure	oks_dump --path "onlsw_test_3x3_lxplus@Partition" "(path-to \"lxplus-3x3-21-ctrl@RunCo ntrolApplication\" (direct \"Segments\" \"OnlineInfrastructure\" (nested \"Segments\" (direct \"Applications\" \"IsControlledBy\" \"Resources\")))\" daq/partitions/lxplus_tests.data.xml	Real example from release notes: path-to destination 'id@class', direct = first-level rels, nested = further levels.
42	What config did run 380689 use?	oks_clone_repository --version tag:r380689@all_hosts	Version selector 'tag:@' for git-backed OKS.
43	What config is at git commit 6800fe3b?	oks_clone_repository --version hash:6800fe3b	Version selector 'hash:'
44	Config as of 2020-08-02 10:00:00	oks_clone_repository --version 'date:2020-08-02 10:00:00'	Version selector 'date:YYYY-MM-DD HH:MM:SS' - quote in shell due to space.
45	Programmatically list all stored versions of a repository	repo.get_repository_versions()	C++ API OksRepository::get_repository_versions; returns list of version strings (used to pick a version selector).
46	Show summary of a repository (run-number tags)	oks_utils rn_ls (example)	Release notes show rn_ls/rn_dump style tools in run-db context; versions can be listed and referenced.
47	GUI: build the same query visually	Data Editor -> Class window -> right-click -> Query	The Query window builds attribute/relationship/logical expressions; saved query files are editable text.
48	Are queries class-strongly-typed?	No - weak typing: same attribute name may match in several classes	Quote from QueryWindow.html: a query does not strongly depend on class type.
49	Find partitions with Max Time <= 2 days (172800 s)	(all ("Max Time" "172800" <=))	Illustrative attr name from TDAQ schemas (time in seconds).
50	Find objects of class 'Alarm' with severity >= 2	(all ("Severity" "2" >=))	Illustrative: TDAQ has Alarm config in DAQ-Configuration schema (captured).

9. Academic and Technical Literature

9.1 CHEP 2008 Configuration Service Paper

Title: The ATLAS DAQ system online configurations database service challenge

Authors: J. Almeida et al. (incl. I. Soloviev)

Source: J. Phys. Conf. Ser. 119 022004 (2008), CHEP 2007

URL: <https://iopscience.iop.org/article/10.1088/1742-6596/119/2/022004>

Local copy: 957,655 B, 12 pages, 43,810 characters of text

Content, as useful for the corpus:

- The configuration service is part of ATLAS DAQ/HLT + Detector Controls; it stores which parts of ATLAS participate in a run, when/where processes start, run-time environments, status checks and recovery, shutdown order, and parameters for data-flow/trigger/detector modules.
- Accessed by thousands of processes concurrently at run start; supports notification and reconfiguration mid-run; archives used configurations for later browsing by experts and for offline event processing.
- OKS is described as an object manager — unique object identifiers, classes/attributes/relationships, composite objects, integrity constraints, dynamic class/object creation, native C++ API, its own query language, and change notifications.
- Storage: human-readable XML files (schema + data, with includes); relational back-ends via CORAL (Oracle/MySQL/SQLite) for incremental archiving; RDB CORBA servers for remote access with query-result caching.
- Editors: a UML-like Schema Editor and a Data Editor with a graphical query constructor whose output is a reusable query file.
- Archiving: the configuration used at the start of each run is archived and made browsable by time, release, user, host, and partition.

Verbatim DAL usage example given in the paper (also referenced in Section 5):

```
Configuration db("oksconfig:daq/partitions/test.xml");
const Partition * format = dal::core::get_partition(db, "test");
const Computer * h = p->get_DefaultHost();
std::cout << "Default host: " << h->UID() << std::endl;
const Computer * h2 = db.get<Computer>("pc-tdaq-onl-02");
p->set_DefaultHost(h2);
db.commit();
```

9.2 L1Calo Database User Guide

Title: DatabaseUserGuide — L1Calo (Trigger Calorimeter) configuration with OKS

Source: Queen Mary, University of London (PPRC):
pprc.qmul.ac.uk/l1calo/doc/out/DatabaseUserGuide.pdf

Local copy: 159,523 B, 22 pages, 44,155 characters

- The L1Calo DB is organised as CMT packages dbFiles (schema+data): the schema defines classes shared by all L1Calo readout modules; data files hold concrete values such as thresholds, formats, dead-time and cable maps.
- Confirms the generic layering: OKS = generic storage layer, DAL = typed access layer generated from the schema; the database lives as .schema.xml/.data.xml under a TDAQ_DB_PATH area.

- Explains how DAL subclasses existing classes (e.g. CaloPprModule → L1Calo-specific classes) to add detector-specific attributes without touching the core schema — the same pattern used generically across the deA standard schema.

9.3 Papers Not Retrievable

A number of other candidate references were attempted and could not be captured; they are listed here for completeness / to document the corpus's known gaps.

Ref	Source	Status
p2	EPJ Web Conf. (2025), epj-conferences.org	403 (Anubis bot wall)
p3	EPJ Web Conf. (2021), epj-conferences.org	403 (Anubis bot wall)
p4	IDIAP (idiap.ch)	404 Not Found
p5	METU (open.metu.edu.tr) — OKS-related thesis	Timeout (connection unreachable)
p6–p10	CERN CDS records 446327, 1457575, 2919574, 2649073, 2923938 — OKS/talk slides	Blocked by Anubis anti-bot page; record page truncates and needs JS challenge

Note on the CDS records: the record numbers (e.g. 446327 — 'OKS - the object kernel for DAQ applications', I. Soloviev, CERN-OPEN-2014) are legitimate, but the server requires a JS challenge; the PDFs can in principle be fetched with a real browser afterward.

10. CERN Indico Tutorials and Talks

10.1 DAQ for Experts Lecture — 'Config Service' Session (Best Shifter Source)

Event: indico.cern.ch/event/106368 (DAQ for Experts school/lecture series). Local PDF: 3,966,147 B, 59 pages.

- The configuration database is called the 'OKS' database in everyday ATLAS jargon; the term originates from the OKS object kernel/manager product.
- Configuration is structured as partitions / segments / applications; each application object describes what to start, where (host/computer), with which environment/command line, and how to recover it.
- Runs are defined by a partition; `setup_daq <partition-file> <name>` starts a session bound to a config version (git hash).
- Includes worked examples of 'which version did this run use?' and 'where is application X configured?' — good question/answer seeds for training data.

10.2 OKS Configuration Service Talk (Information Service, CHEP era)

Event: Indico, CHEP 2007 conference, contribution 332. Local PDF: 1,195,401 B, 14 pages. Content: I. Soloviev et al., 'ATLAS DAQ and configuration service' — introduces OKS, DAL, and the config-service architecture; describes archiving of used configurations at the start of every run for later retrieval by experts.

10.3 Data Editor / GUI Training ('INFOST-2016')

Event: indico.cern.ch/event/548007. Local PDF: 22,603,975 B, 97 pages (mostly screenshots, scanned); text-layer extract is 36,469 characters and contains the step-by-step layout of training on `oks_data_editor`.

10.4 Global Monitoring Overview (Run-Dependent Info)

Event: Indico (Global Monitoring overview). Local PDF: 1,208,147 B, 42 pages. Explains run number, partitions, and how COOL stores configuration per run offline; confirms the split 'online → OKS (real-time config), offline → COOL (by run).' Not OKS-heavy, but supplies vocabulary for shifter questions.

10.5 Poster — Bianchi, EPS 2011

Local PDF: 5,588,158 B, 1 page; extract 7,821 characters. 'The ATLAS configuration service' poster; states the OKS/DAL/config layering and describes how run-relevant config data is archived — a good short citation source.

10.6 Sessions Not Available

Event	Content	Reason
indico.cern.ch/event/1452600	DAQ for Experts — Run 3 (modern)	Requires access key (private/restricted)
agenda.infn.it/.../TDAQ_BO_Ep_2.pdf	DAQ bootcamp EP2	Anubis anti-bot; direct .pdf path 404s
indico.cern.ch/event/1566094/contribution/6690058	'AI RCS Strategy Workshop: AI Assistant for ATLAS operations and beyond' (Sep 2025)	Accessible page but no attached materials (abstract only)

The AI RCS Strategy Workshop abstract is worth flagging specifically: it confirms that the planned 'DAQ Shifter Assistant' feature set includes config-version lookups and query building — directly validating the motivation for this harvest and the associated minor-project agent (see profile note).

11. Original CERN GitLab Sources

All repositories below are public at <https://gitlab.cern.ch> (no login needed for browsing; anonymous git clone works over https):

Repo	URL	Size
oks	gitlab.cern.ch/atlas-tdaq-software/oks	56 files
oks_utils	gitlab.cern.ch/atlas-tdaq-software/oks_utils	260 files
swrod	gitlab.cern.ch/atlas-tdaq-software/swrod	227 files
oks2coral	gitlab.cern.ch/atlas-tdaq-software/oks2coral	16 files

Plus the public release-notes site: <https://atlas-tdaq-sw-releases.web.cern.ch/>.

11.1 oks (CERN Original)

`doc/RELEASE_NOTES.md` is the central document, covering `tdaq-13-01-00` through `tdaq-01-01-00` (query/archiving/git facts, summarised in Section 6). `include/oks/*.h` are the canonical headers (the DUNE fork branches from tag `oks-08-03-04`). `test/` holds `test_update.cpp` (how to update attributes/relations), `test.data.xml`, `all_types.schema.xml`. `scripts/*.sh` holds `oks-commit.sh`, `oks-tag.sh`, `oks-checkout.sh`, `oks-import.sh`, `oks-copy.sh`, `oks-edit-branch.sh`. `jsrc/` contains a browse-only Java OKS mirror.

11.2 oks_utils (Tools, Editors, Archiving)

`examples/` holds the tutorial programs `query.cpp`, `comparator.cpp`, `and_expression.cpp`, `or_expression.cpp`, `not_expression.cpp`, `r_expression.cpp`, `index.cpp`, `kernel.cpp`, `object.cpp`, `relationship.cpp`, `class.cpp`, `attribute.cpp`, `method.cpp`, `data.cpp`, `alloc.cpp`, `profiler.cpp` (quoted extensively in Section 3–4). `src/bin/oks_tutorial.cpp` is a self-contained tutorial building a 'Car/Person' database and running queries. `src/xm-gui/` holds the OKS Data Editor (Motif) sources including query-constructor dialogs. `data/online-help/data-editor/*.html` is the GUI user manual (54 files, copied locally) — `QueryWindow.html`, `ObjectWindow.html`, `DataFileWindow.html`, `ClassWindow.html`, `MainWindow.html`, `GraphicalWindow.html`, `ReplaceWindow.html`, `OksDataEditor.html`, of which **QueryWindow.html documents the query builder** in full (attribute expressions, relationship expressions, and the and/or/not tree). `src/rlib/` is the relational archiving library plus `create_db.{oracle,mysql,sqlite}.sql`. `cgi/` holds `oks-archive.pl` (web archive GUI), `getdata.sh`, `cmp_data_versions.sh`, `vtable.sh`.

11.3 swrod — A Modern Consumer of OKS (Shifter-Visible Classes)

`schema/swrod.schema.xml` (23.6 KB) is the SW ROD config schema, defining classes `SwRodApplication`, `SwRodConfiguration`, `SwRodRob`, `SwRodModule`, `SwRodInputLink`, `SwRodDataChannel`, `SwRodCustomProcessingLib`, `SwRodFragmentBuilder`, `SwRodGBTModeBuilder`, `SwRodFullModeBuilder`, `SwRodFelixInput`, `SwRodBufferInput`, `SwRodFileWriter`, `SwRodHLTRequestHandler`, `SwRodEventSampler`, `SwRodDebugStream`, `SwRodCustomProcessor`, `SwRodShredder`, `FelixLink`, and more. `data/*.data.xml` holds realistic configuration values (ROB, input links per DMA, hosts, modules); `application/Configuration.cpp` + `main.cpp` show how a run-control FSM application reads the OKS configuration at startup.

11.4 oks2coral — Archiving Bridge

src/oks2coral.cpp is a CLI that checks schema/data into an Oracle 'OKS archive' (versions, tags). data/oks-archive-info.xml is the ConfigVersions schema class used to record 'this run used version X' — reproduced in full in Section 7.5.

11.5 Release Notes — Facts Pinned Down for the Corpus

(Summarised in detail in Section 6; sourced from oks-atlas/doc/RELEASE_NOTES.md.)

11.6 Release-Notes Web Site

<https://atlas-tdaq-sw-releases.web.cern.ch/> — an MkDocs site listing releases tdaq-09-00-00 through tdaq-13-00-00 plus nightly builds; points to gitlab.cern.ch/atlas-tdaq-software/admin/tdaq-release-notes.

11.7 Tags in the oks GitLab Repository

Retrieved via `GET /api/v4/projects/atlas-tdaq-software%2Foks/repository/tags`. Latest tags: oks-08-04-00 (tdaq-11-02-00, Nov 2023), oks-08-03-04 (tdaq-09-04-00, 2022 — **the DUNE fork baseline**), oks-08-03-03 (tdaq-09-03-00), oks-08-03-02-01, ... down to oks-08-00-00 (the git migration point) and oks-07-00-08 through 06-10-xx (the CVS era).

12. Shifter-Facing Documentation

This section collects the sources written for ATLAS shifters/operators — i.e. the 'user' side of OKS: what is stored, how to answer 'which configuration was used', and how to browse the database without touching the C++/Python API.

12.1 Public Sources (Accessible)

12.1.1 DBE (DataBase Editor) User Manual — *ReadTheDocs Branch*

URL: dune-daq-sw.readthedocs.io/en/johnfreeman-update_documentation_instructions/packages/dbe/. DBE is the OKS DataBase Editor, the run-control GUI used to browse and edit the configuration database (classes, objects, includes, comments; check-in/check-out/commit; versioning). Related sub-page: .../packages/dbe/schemaeditor/.

12.1.2 OKS Data Editor Online Help (from *oks_utils*, copied locally)

Local copy: [repos/online-help/data-editor/](#) (54 files including GIF screenshots). [QueryWindow.html](#) is the full manual of the OKS query constructor (attribute expression, relationship expression, logical and/or/not expressions, saving/loading query files in the LISP-like text format). Index page [OksDataEditor.html](#) links out to [MainWindow.html](#), [ClassWindow.html](#), [DataFileWindow.html](#), [ObjectWindow.html](#), [GraphicalWindow.html](#), [ReplaceWindow.html](#), [MessageLogWindow.html](#).

12.1.3 NEVIS Wiki — *FLX Configuration (Public Foswiki)*

URL: wiki.nevis.columbia.edu/bin/view/FLX_Configuration. A FELIX configuration summary written for collider operations; includes where the configuration lives in OKS terms and an 'FLX_Configuration' cheat sheet.

12.1.4 Release-Notes and Version Docs

atlas-tdaq-sw-releases.web.cern.ch/ — HOWTOs (containers, python pip, package tags, release-notes format) plus release notes tdaq-09 through tdaq-13.

12.2 Sources That Require Authentication (CERN Accounts)

All of the following are twiki.cern.ch/twiki/bin/view{/viewauth/}Atlas/... pages, which returned **HTTP 401 (requires authentication)** to anonymous fetching. They are listed because docs and code cross-reference them and they are likely the most authoritative shifter material:

- [DaqHltOks](#) — the master OKS page: architecture, query language user guide, section '4. OKS Git Repository' (linked from [RELEASE_NOTES tdaq-09-01-00](#)).
- [DaqHltConfig / DaqHltConfigService](#) — configuration service how-to.
- [DaqHltShifter / DaqHltShifters](#) — run-shift instructions, including 'what config was this run?' (page names as referenced; to be confirmed after TWiki login).
- [DaqHltAccessManager](#), [SWRodInputErrors](#), [ROBFragmentHeaderStatusWords](#), [DaqHltDal](#) (DAL class docs; the [core.schema.xml](#) descriptions carry links of the form twiki.cern.ch/twiki/bin/viewauth/Atlas/DaqHltDal#3_1_Partition_Class).
- [Atlas/AtlasOnline](#) (the parent collaboration index page).

To retrieve them: authenticate on twiki.cern.ch via SSO, then import pages (e.g. `curl -b cookies`). This remains the main gap for shifter-specific material; the corpus marks these with a placeholder pattern:

[Q] What configuration was used for run N?

[A] <twiki DaqHltShifter excerpt> ; OKS answer: tag r<N>@<partition>, see Sections 4/11.

12.3 Gap Analysis — What Is Missing for a 'Perfect' Shifter Manual

- Authenticated TWiki pages (Section 12.2).
- CDS records blocked by the Anubis anti-bot wall (Section 9.3) — notably CERN-OPEN-2014 'OKS'.
- Indico event 1452600 'DAQ for Experts (Run 3)' — access-key restricted; potentially the best modern shifter primer. Recommendation: obtain a key, or build a training subset from the TWiki mirror instead.
- The 'AI Assistant for ATLAS operations and beyond' workshop (Indico 1566094 / contribution 6690058) is public but has no attached materials; its abstract nonetheless confirms the planned DAQ Shifter Assistant features — config-version lookups and query building — a good scenario seed for this project's own agent (see the CERN-agent minor-project proposal).

13. Source Access Status (Captured vs Blocked)

Compiled 2026-08-08 during the scraping of the DUNE-DAQ oks/dal and ATLAS TDAQ sources; this ledger records exactly what was captured, what was blocked, and what remains untried.

13.1 Fully Accessible and Captured

Material	Where captured	Notes
DUNE oks source (query.cpp, oks_dump, release info)	Section 4	GitHub, public
DUNE dal + tutorial schema/data + tutorial.py	Section 5	GitHub, public
Extra schemas (swrod, core, all_types, DAQ-Configuration, default-parameters, dal_testing)	Section 7 (worked examples)	GitHub, public
CERN original oks GitLab + release notes + tags API	Section 11	gitlab.cern.ch, public
OKS Data Editor online help (all windows incl. QueryWindow)	Sections 3, 12.1.2	kept as captured copies
Papers: CHEP-2008 config-service & DAL, QMUL L1Calo DB guide	Section 9	DOI/IOP public
Indico talks: DAQ-for-experts, Bianchi EPS-2011 poster, ID training 2016, GM overview, talk 332	Section 10	CERN Indico public
Docs/README.md of oks (DUNE fork)	Sections 2, 5	GitHub

13.2 Blocked / Not Retrievable

Item	Reason	Workaround used
EPJS paper 403 (EPJ paper mirror)	403 / failed mirror	Skipped; noted in Section 9; content substituted from DAL examples + guides
CDS/Anubis config-service papers	Records old, attachments blocked to bots	Noted in Section 9.3
ATLAS TWiki (atlas-tdaq-info pages: ShifterDocs)	Requires CERN SSO	Excluded; DBE/online-help used instead (Section 12)
Some Indico events (notes/slides for old CHEP talks)	ACL-restricted or missing PDF	Only abstracts listed (Section 10.6)
p1 IOP 119,022004 full text of a related paper	—	PDF fetched successfully; extracted (Section 9.1)

13.3 Not Yet Tried (Worth a Later Pass)

- atlasop.cern.ch/atlas-point1/tdaq/web_is/app/oks.html (requires an ATLAS Point-1 account).

- A fresh Anubis-aware pass for oks2coral and swrod provenance, if ever needed for fine-tuning YAML.
- DUNE's daq_config repository (the successor to oks) — not needed for OKS-query fine-tuning, but relevant if the schema evolves further.

14. Final Deliverables Summary and Corpus Statistics

Goal restated: produce a comprehensive corpus about the OKS configuration database and its query language, for use as reference material when writing a DUNE-DAQ OKS-query fine-tuning dataset (English question → OKS query string). Harvest date: 2026-08-08.

14.1 Deliverable Files

File	Content
01_dune_oks_code.md	DUNE-DAQ/oks fork: query grammar (this/all, and/or/not, some/all, comparators, object-id), query classes, oks_dump options/exit codes, git-versioning API, Python bindings
02_dune_dal_code.md	DUNE-DAQ/dal: directory layout, core/tutorial schema, tutorial data + tutorial.py, ConfigVersion & IS pattern docs
03_papers.md	CHEP-2008 config-service (DAL) paper; L1Calo DB guide; status of EPJ-403 / CDS queries (blocked)
04_indico_tutorials.md	Five captured Indico PDFs with page maps; restricted events listed
05_cern_gitlab.md	Public CERN GitLab: oks, oks_utils, swrod, oks2coral; release-notes facts; GitLab tags API sample output
06_shifter_manuals.md	Shifter-facing corpus: DBE page, OKS Data Editor online help, Nevis wiki notes; TWiki gap flagged
oks_schema_examples.xml	5 annotated schema/data examples (Section 7)
oks_query_examples.md	Grammar BNF, English→query table, git-version queries, comparator enum, GUI semantics, errors/exit codes
gold_pairs.jsonl	50-row gold-standard {question, query_oks, note, source_file} corpus (Section 8)
access_status.md	Captured vs blocked vs not-yet-tried ledger (Section 13)

14.2 Raw Captures Retained

- pdfs/ — the p1 IOPS paper + extract, the p11 QMUL L1Calo guide + extract.
- indico/ — five PDFs plus text extracts.
- repos/ — original GitHub/CERN files: schemas (tutorial, test.data, swrod, core, all_types, DAQ-Configuration, default-parameters, dal_testing), tutorial.py, and the full Data Editor online-help tree.

14.3 Facts Pinned Down (Worth Gold-Labelling)

- Query file format is plain text, Lisp-like; the GUI can save/load query files ('the format of the query file is simple ... edit manually by any text editor').
- (this ...) = class-only vs (all ...) = class + subclasses.
- some = at least one referenced object matches; all = every referenced object matches.
- Only = is allowed when comparing object-id.

- `~=` (regex compare) was added in OKS tdaq-01-09-00.
- Values are always quoted strings; type interpretation (numeric vs string) is weak and only matters at runtime.
- `oks_dump` exit codes: 0 ok, 1 CLI, 2 file, 3 query, 4 class, 5 dangling references.
- Git versioning selectors: `tag:r380689@all_hosts`, `hash:`, `date:...` in `oks_clone_repository`; `get_repository_versions()`.
- Path-query syntax (`path-to ... (direct ... nested ...)`) verified from both the OKS sources and `RELEASE_NOTES` tdaq-01-02-00.

14.4 Gaps / Risks

- The authoritative specification is prose (README, QueryWindow.html, RELEASE_NOTES) — no formal grammar file exists; the BNF in Section 3.1 is reconstructed and should be kept labelled as such.
- TWiki and atlasop pages require CERN SSO and were not harvested.
- One attempted paper (EPJ-403) is not retrievable; the L1Calo guide plus the DAL chapter of the CHEP paper serve as substitute grounding where needed.

14.5 Suggested Next Steps (as Recorded in the Source Report)

- *Done*: built a gold set of question/query pairs from `oks_query_examples.md` — delivered as `gold_pairs.jsonl` (50 rows), covering all comparators, this/all scopes, and/or/not, relationship some/all, object-id, nested relationships, path-to queries, version selectors, `oks_dump` CLI/exit codes, and GUI/weak-typing facts (Section 8).
- Use `oks_dump -f <file> -c <cls> -q '<query>'` on the copied `repos/*.data.xml` files to validate each pair end-to-end against a real OKS engine.
- Optionally ship a final fine-tuning JSONL using the schema `{question, query_oks, note, source_file}`.

15. Appendix A — Full Reserved Token / Comparator Reference

Reserved tokens (src/query.cpp, lines 23-39):

or	"or"	
and	"and"	
not	"not"	
some	"some"	
this	"this"	(scope: class only)
all	"all"	(scope: class + all subclasses)
object-id	"object-id"	
=	equal	
!=	not equal	
~=	regex match	(added tdaq-01-09-00)
<=	less or equal	
>=	greater or equal	
<	less than	
>	greater than	
path-to	"path-to"	(path query root)
direct	"direct"	(path query: direct relationship hop)
nested	"nested"	(path query: nested/recursive hop)

Comparator enum -> operator text (include/oks/query.hpp):

OksQuery::equal_cmp	=
OksQuery::not_equal_cmp	!=
OksQuery::less_cmp	<
OksQuery::less_or_equal_cmp	<=
OksQuery::greater_cmp	>
OksQuery::greater_or_equal_cmp	>=
(regex comparator, index.hpp)	~=

oks_dump exit codes:

0	OK
1	Bad command line
2	Bad OKS file(s)
3	Bad query
4	Class not found
5	Dangling references found

Git version selectors (oks_clone_repository --version ...):

```
tag:r380689@all_hosts
hash:6800fe3b
date:'2020-08-02 10:00:00'
```

16. Appendix B — Provenance Map

Cross-reference of key facts to their exact source location, reproduced from `oks_query_examples.md`:

Fact	Source
Grammar tokens & comparators	<code>src/query.cpp</code> lines 23-39; <code>include/oks/query.h</code>
object-id example	CERN RELEASE_NOTES tdaq-01-02-00, section 'Query', subsection 'Example of query string'
Path-query example	CERN RELEASE_NOTES tdaq-01-02-00, 'Path Query'; <code>oks_dump --path</code> in the same document
Regex comparator <code>~=</code>	RELEASE_NOTES tdaq-01-09-00 ('add attribute comparator <code>'~='</code>)
Timing / versioning	RELEASE_NOTES tdaq-09-02-01 + tdaq-09-01-00 (<code>rn_ls</code> example)
GUI semantics	<code>repos/online-help/data-editor/QueryWindow.html</code>

End of report. All content in this document is derived from the eleven source files listed in Section 14.1, which were compiled during the 2026-08-08 OKS/DAL harvest; no material from those files has been omitted.